

DELHIVERY FEATURE ENGINEERING & LOGISTICS ANALYTICS REPORT

1. Executive Summary

This project focuses on transforming raw logistics shipment data from Delhivery into a structured analytical dataset suitable for forecasting, operational optimization, and future machine learning applications. The original dataset consisted of shipment-level and segment-level records, where a single shipment could span multiple transportation segments.

The project involved data cleaning, aggregation, feature engineering, exploratory data analysis, hypothesis testing, outlier treatment, categorical encoding, and feature scaling. Through statistical analysis and business-focused exploration, several operational inefficiencies were identified, including significant differences between actual delivery times and OSRM route estimates, high-delay logistics corridors, and substantial variability in delivery performance.

The final output is a machine-learning-ready dataset enriched with engineered features and actionable business insights that can support ETA prediction, route optimization, and logistics planning.

2. Problem Statement

Delhivery generates large volumes of logistics movement data through its transportation network. Since shipment information is distributed across multiple route segments, direct analysis of the raw data does not provide meaningful business insights.

The objective of this project was to:

- Aggregate segment-level records into trip-level observations.
 - Create useful features from raw operational data.
 - Validate relationships between route estimates and actual delivery performance.
 - Identify delivery bottlenecks and inefficiencies.
 - Prepare the dataset for future forecasting and machine learning applications.
-

3. Dataset Understanding and Initial Exploration

The dataset contained 144,867 records and 24 features related to shipment movement, route estimates, actual delivery performance, and transportation routes.

Initial exploration was performed to understand:

- Dataset dimensions
- Feature types
- Missing values
- Data quality issues
- Aggregation requirements

df.describe()					
	start_scan_to_end_scan	cutoff_factor	actual_distance_to_destination	actual_time	osrm_ti
count	144867.000000	144867.000000	144867.000000	144867.000000	144867.000000
mean	961.262986	232.926567	234.073372	416.927527	213.8681
std	1037.012769	344.755577	344.990009	598.103621	308.0111
min	20.000000	9.000000	9.000045	9.000000	6.000000
25%	161.000000	22.000000	23.355874	51.000000	27.000000
50%	449.000000	66.000000	66.126571	132.000000	64.000000
75%	1634.000000	286.000000	286.708875	513.000000	257.000000
max	7898.000000	1927.000000	1927.447705	4532.000000	1686.000000

```
df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 144867 entries, 0 to 144866
Data columns (total 24 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   data                                  144867 non-null object
1   trip_creation_time                    144867 non-null object
2   route_schedule_uuid                   144867 non-null object
3   route_type                            144867 non-null object
4   trip_uuid                             144867 non-null object
5   source_center                         144867 non-null object
6   source_name                           144574 non-null object
7   destination_center                    144867 non-null object
8   destination_name                       144606 non-null object
9   od_start_time                         144867 non-null object
10  od_end_time                           144867 non-null object
11  start_scan_to_end_scan                 144867 non-null float64
12  is_cutoff                             144867 non-null bool
13  cutoff_factor                          144867 non-null int64
14  cutoff_timestamp                       144867 non-null object
15  actual_distance_to_destination         144867 non-null float64
16  actual_time                            144867 non-null float64
17  osrm_time                              144867 non-null float64
18  osrm_distance                          144867 non-null float64
19  factor                                144867 non-null float64
20  segment_actual_time                    144867 non-null float64
21  segment_osrm_time                      144867 non-null float64
22  segment_osrm_distance                  144867 non-null float64
23  segment_factor                         144867 non-null float64
dtypes: bool(1), float64(10), int64(1), object(12)
memory usage: 25.6+ MB
```

INITIAL EXPLORATION

df.shape									
(144867, 24)									
df.head()									
	data	trip_creation_time	route_schedule_uuid	route_type	trip_uuid	source_center	source_name	destination_center	destination_name
0	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)
1	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)
2	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)
3	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)
4	training	2018-09-20 02:35:36.476840	thanos::sroute:eb7bfc78-b351-4c0e-a951-fa3d5c3...	Carting	153741093647649320	IND388121AAA	Anand_VUNagar_DC (Gujarat)	IND388620AAB	Khambhat_MotvdDPP_D (Gujarat)

Observations

- The dataset contained both numerical and categorical features.
- Several datetime variables required conversion before analysis.
- Shipment records existed at the segment level, requiring aggregation before business-level analysis could be performed.

Insight

The structure of the dataset confirmed that feature engineering and aggregation would be essential before any meaningful analytical work could begin.

4. Data Cleaning and Preprocessing

The first stage of preprocessing focused on identifying missing values and validating data quality.

Missing value analysis revealed:

- source_name: 293 missing values
- destination_name: 261 missing values

The percentage of missing observations was relatively small compared to the overall dataset size.

Datetime conversion was performed on:

- trip_creation_time
- od_start_time
- od_end_time
- cutoff_timestamp

CONVERTING DATE ENTRIES FROM OBJECT TO DATE TYPE

```
: date_cols=["trip_creation_time","od_start_time","od_end_time","cutoff_timestamp"]
:
: for col in date_cols:
:     df[col]=pd.to_datetime(df[col],format='mixed')
:
: df[date_cols].dtypes
:
: trip_creation_time    datetime64[ns]
: od_start_time        datetime64[ns]
: od_end_time          datetime64[ns]
: cutoff_timestamp     datetime64[ns]
: dtype: object
```

```
|: df.isnull().sum()
|:
|: data                                0
|: trip_creation_time                 0
|: route_schedule_uuid                0
|: route_type                         0
|: trip_uuid                          0
|: source_center                      0
|: source_name                        293
|: destination_center                 0
|: destination_name                   261
|: od_start_time                      0
|: od_end_time                        0
|: start_scan_to_end_scan             0
|: is_cutoff                          0
|: cutoff_factor                      0
|: cutoff_timestamp                   0
|: actual_distance_to_destination     0
|: actual_time                       0
|: osrm_time                         0
|: osrm_distance                     0
|: factor                             0
|: segment_actual_time                0
|: segment_osrm_time                  0
|: segment_osrm_distance              0
|: segment_factor                     0
|: dtype: int64
```

Observations

- Missing values represented a very small portion of the dataset.
- Datetime conversion enabled the creation of operational duration features.
- Data quality was generally good, requiring minimal cleaning.

Insight

The dataset was sufficiently complete to proceed with aggregation and feature engineering without significant information loss.

5. Aggregation Strategy

Since a single shipment may contain multiple route segments, aggregation was necessary to create trip-level observations.

Aggregation was performed in two stages:

Stage 1

Grouped by:

- trip_uid
- source_center
- destination_center

Stage 2 (144,867 to 14807 records)

Further aggregated by:

- trip_uid

Numerical variables such as actual_time, osrm_time, and distance-related metrics were aggregated using summation, while categorical variables used first or last occurrences depending on business relevance.

AGGREGATION AT CORRIDOR LEVEL

```
corridor_df=df.groupby(['trip_uid','source_center','destination_center']).agg(  
    {  
        'trip_creation_time':'first',  
        'route_type':'first',  
        'source_name':'first',  
        'destination_name':'last',  
        'actual_time':'sum',  
        'osrm_time':'sum',  
        'segment_actual_time':'sum',  
        'segment_osrm_time':'sum',  
        'osrm_distance':'sum',  
        'segment_osrm_distance':'sum',  
        'actual_distance_to_destination':'last',  
    }  
).reset_index()
```

corridor_df

	trip_uid	source_center	destination_center	trip_creation_time	route_type	
0	trip-153671041653548748	IND209304AAA	IND000000ACB	2018-09-12 00:00:16.535741	FTL	Kanpur
1	trip-153671041653548748	IND462022AAA	IND209304AAA	2018-09-12 00:00:16.535741	FTL	Bh
2	trip-153671042288605164	IND561203AAB	IND562101AAA	2018-09-12 00:00:22.886430	Carting	Doddablp
3	trip-153671042288605164	IND572101AAA	IND561203AAB	2018-09-12 00:00:22.886430	Carting	Tur

AGGREGATION AT TRIP LEVEL

```
trip_df = corridor_df.groupby('trip_uid').agg(  
    {  
        'trip_creation_time':'first',  
        'route_type':'first',  
        'source_name':'first',  
        'destination_name':'last',  
        'actual_time':'sum',  
        'osrm_time':'sum',  
        'segment_actual_time':'sum',  
        'segment_osrm_time':'sum',  
        'osrm_distance':'sum',  
        'segment_osrm_distance':'sum',  
        'actual_distance_to_destination':'sum',  
    }  
).reset_index()
```

trip_df

	trip_uid	trip_creation_time	route_type	source_name	destin
0	trip-153671041653548748	2018-09-12 00:00:16.535741	FTL	Kanpur_Central_H_6 (Uttar Pradesh)	Kanpur (Ut
1	trip-153671042288605164	2018-09-12 00:00:22.886430	Carting	Doddablpur_ChikaDPP_D (Karnataka)	Doddablpur
2	trip-153671043369099517	2018-09-12 00:00:33.691250	FTL	Gurgaon_Bilaspur_HB (Haryana)	Gurgaon
3	trip-153671046011330457	2018-09-12 00:01:00.113710	Carting	Mumbai Hub (Maharashtra)	Mumbi
4	trip-153671052974046625	2018-09-12 00:02:09.740725	FTL	Bellary_Dc (Karnataka)	Sandur_W

Observations

- The dataset was reduced from 144,867 records to approximately 14,807 unique trips.
- Aggregation successfully consolidated fragmented shipment information.

Insight

Trip-level aggregation enabled accurate measurement of delivery performance, route efficiency, and corridor-level operational behavior, which would not have been possible using the raw segment-level data..

6. Feature Engineering

Several new features were created to enrich the dataset.

Location-Based Features

Extracted:

- Source State
- Destination State

```
EXTRACTING SOURCE AND DESTINATION STATE
: trip_df['source_state']=trip_df['source_name'].str.extract(r'\((.*?)\)')
: trip_df['destination_state']=trip_df['destination_name'].str.extract(r'\((.*?)\)')
: trip_df['source_state']

: 0      Uttar Pradesh
  1      Karnataka
  2      Haryana
  3      Maharashtra
  4      Karnataka
  ...
14812    Punjab
14813    Haryana
14814    Uttar Pradesh
14815    Tamil Nadu
14816    Karnataka
Name: source_state, Length: 14817, dtype: object
```

Time-Based Features

Derived:

- Month
- Day
- Year
- Weekday

```
CREATING FEATURES FOR ANALYSIS
: trip_df['month'] = trip_df['trip_creation_time'].dt.month
: trip_df['year'] = trip_df['trip_creation_time'].dt.year
: trip_df['day'] = trip_df['trip_creation_time'].dt.day
: trip_df['weekday'] = trip_df['trip_creation_time'].dt.day_name()
```

Operational Features

Created:

- OD Duration
- Corridor Feature

```
CALCULATING TRIP TIME / OD TIME
]: df['od_duration']=(df['od_end_time'] - df['od_start_time']).dt.total_seconds()/60
```

Observations

- Feature engineering transformed raw location strings into meaningful business attributes.
- Time features enabled temporal analysis.
- Corridor creation enabled route-level performance evaluation.

Insight

The newly created features enabled identification of shipment concentration patterns, route-level bottlenecks, and state-to-state corridor performance..

7. Validation of Operational Duration Metrics

A new feature called OD Duration was created using `od_start_time` and `od_end_time`.

The objective was to compare this engineered feature with `start_scan_to_end_scan` to determine whether both variables represented similar operational processes.

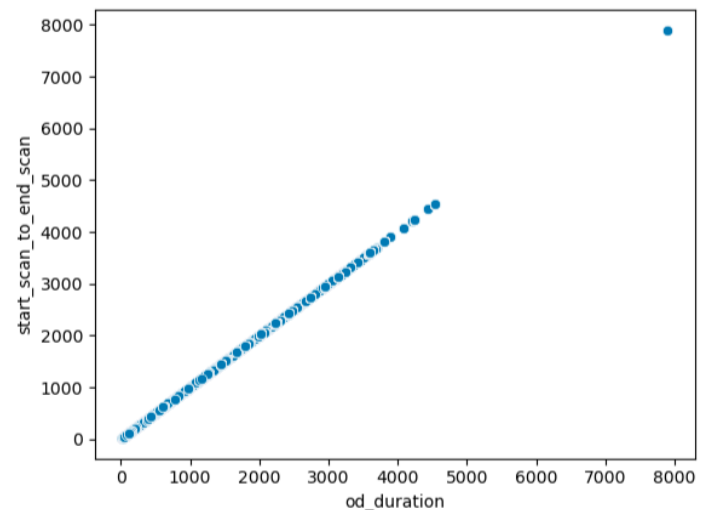
Observations.

- The scatter plot demonstrates a near-linear relationship between the two variables, indicating strong consistency between independently recorded operational timestamps.

Insight

The close alignment confirms that OD Duration is a reliable representation of actual shipment movement duration and can be confidently used in downstream analysis and predictive models.

```
sns.scatterplot(x=df['od_duration'],y=df['start_scan_to_end_scan'])  
plt.show()
```



8. Actual Time vs OSRM Time Analysis

One of the primary objectives of the project was to evaluate the difference between actual delivery performance and route estimates generated by OSRM.

A paired hypothesis test was conducted.

Findings

- Actual delivery times were significantly higher than OSRM estimated times.
- Statistical testing confirmed that the difference was highly significant.

Observations

- Actual delivery time averaged 4,078 minutes which is approximately **95% higher** than OSRM estimated times on average.
- The boxplot comparison shows that actual delivery times exhibit significantly greater variability and more extreme delays than OSRM estimates

Insight

Route travel time alone does not explain delivery performance. Operational delays such as loading, unloading, hub processing, and waiting periods contribute significantly to overall delivery duration.

Statistical Results

- Mean Actual Time = **4,078.8 minutes**
- Mean OSRM Time = **2,092.2 minutes**
- Difference = **1,986.6 minutes**
- T-statistic = **32.47**
- P-value = **1.95×10^{-223}**

```
#COMAPRISON I (ACTUAL TIME VS OSRM TIME)

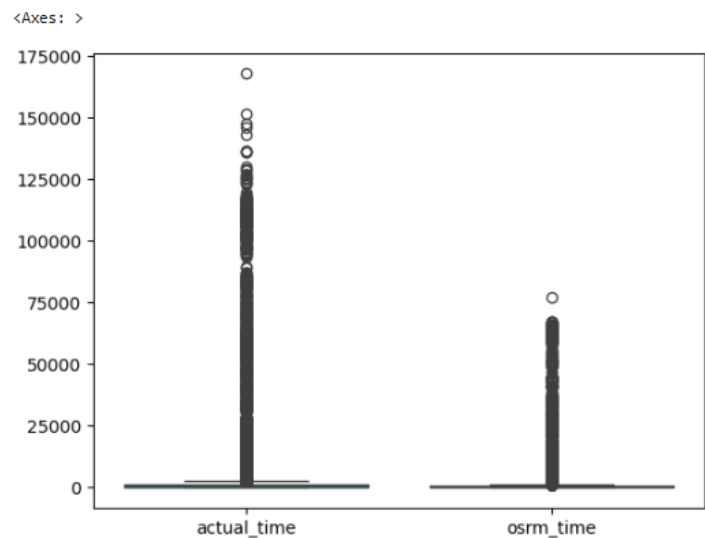
from scipy.stats import ttest_rel

t_stat, p_value = ttest_rel(trip_df['actual_time'], trip_df['osrm_time'])
p_value, t_stat

(np.float64(1.955236157969422e-223), np.float64(32.46688399719858))
```

```
#VISUAL ANALYSIS OF THE ABOVE
```

```
sns.boxplot(data=trip_df[['actual_time', 'osrm_time']])
```



```
trip_df[['actual_time', 'osrm_time']].describe()
```

	actual_time	osrm_time
count	14807.000000	14807.000000
mean	4078.797596	2092.244006
std	15221.711050	7959.425517
min	9.000000	6.000000
25%	142.000000	62.000000
50%	348.000000	167.000000
75%	1064.000000	516.000000
max	167920.000000	76953.000000

9. Actual Time vs Segment Actual Time Analysis

The relationship between trip-level delivery duration and segment-level delivery duration was evaluated.

Findings

- Strong positive correlation was observed.
- Statistical testing indicated a significant difference between the two measures.

Observations

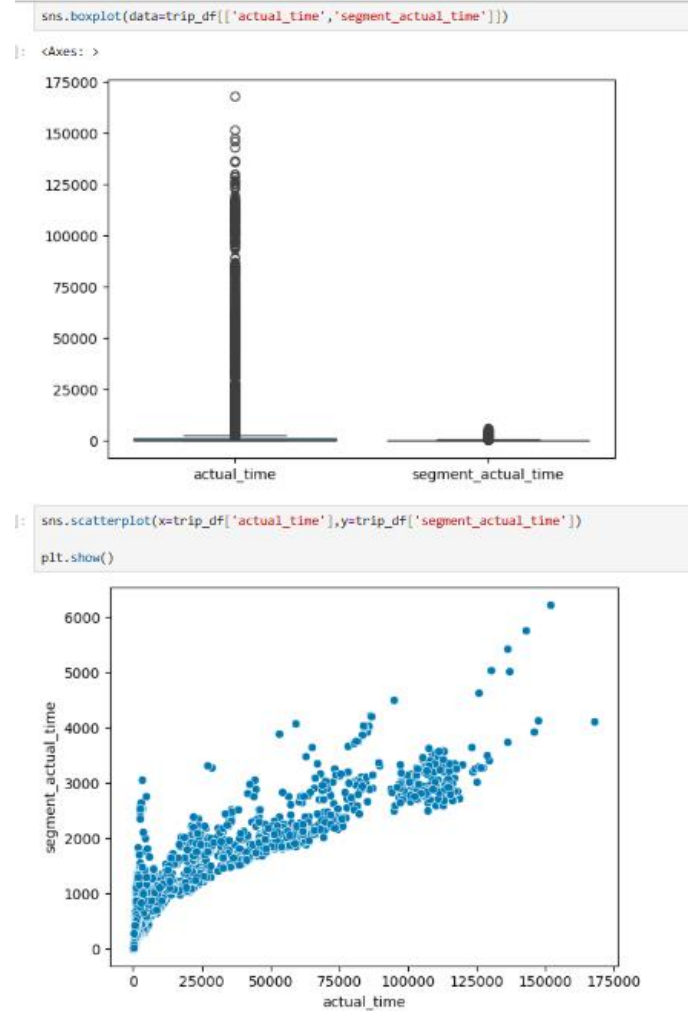
- A correlation of 0.875 indicates a strong positive relationship between segment movement duration and total trip duration.
- However, the significant hypothesis test result indicates that total delivery duration includes substantial additional operational time beyond pure transportation movement.

Insight

Movement time alone is insufficient for accurate ETA prediction. Operational process metrics should also be incorporated into forecasting models.

Statistical Results

- Correlation = **0.875**
- T-statistic = **30.75**
- P-value = **2.16×10^{-201}**



```
ttest_rel(trip_df['actual_time'], trip_df['segment_actual_time'])
```

TtestResult(statistic=np.float64(30.754508844560945), pvalue=np.float64(2.1583615394356942e-201),

```
trip_df[['actual_time', 'segment_actual_time']].describe()
```

	actual_time	segment_actual_time
count	14807.000000	14807.000000
mean	4078.797596	354.031539
std	15221.711050	556.407579
min	9.000000	9.000000
25%	142.000000	66.000000
50%	348.000000	147.000000
75%	1064.000000	367.000000
max	167920.000000	6230.000000

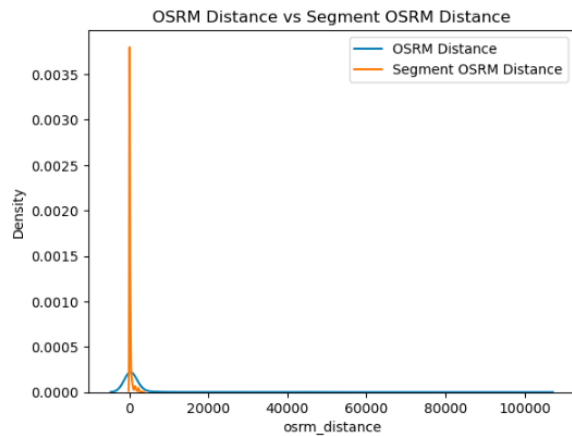
10. OSRM Distance and Time Consistency Analysis

Route-level and segment-level OSRM estimates were compared to validate aggregation quality.

```
#COMPARISON III - (OSRM DISTANCE VS SEGMENT OSRM DISTANCE)

sns.kdeplot(trip_df['osrm_distance'],label='OSRM Distance')
sns.kdeplot(trip_df['segment_osrm_distance'], label='Segment OSRM Distance')

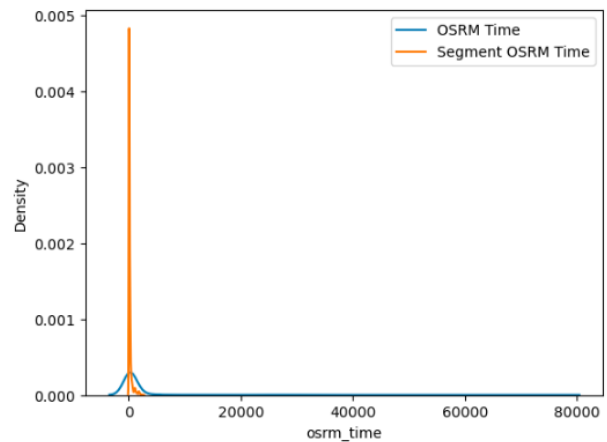
plt.legend()
plt.title("OSRM Distance vs Segment OSRM Distance")
plt.show()
```



```
#COMPARISON IV (OSRM TIME VS SEGMENT OSRM TIME)

sns.kdeplot(trip_df['osrm_time'],label='OSRM Time')
sns.kdeplot(trip_df['segment_osrm_time'],label='Segment OSRM Time')

plt.legend()
plt.show()
```



```
: trip_df[['osrm_distance','segment_osrm_distance']].corr()
:
:      osrm_distance  segment_osrm_distance
:      osrm_distance      1.000000      0.911177
:      segment_osrm_distance      0.911177      1.000000

: ttest_rel(trip_df['osrm_distance'],trip_df['segment_osrm_distance'])
: TtestResult(statistic=np.float64(30.029329602535004), pvalue=np.float64(2.256299014924368e-192), df=
```

```
: trip_df[['osrm_time','segment_osrm_time']].corr()
:
:      osrm_time  segment_osrm_time
:      osrm_time      1.000000      0.903184
:      segment_osrm_time      0.903184      1.000000

: ttest_rel(trip_df['osrm_time'],trip_df['segment_osrm_time'])
: TtestResult(statistic=np.float64(30.296253859676927), pvalue=np.float64(1.136640199143338e-195), df=
```

Findings

- Strong correlations were observed for both distance and time metrics.
- Aggregation preserved route-level information effectively.

Observations

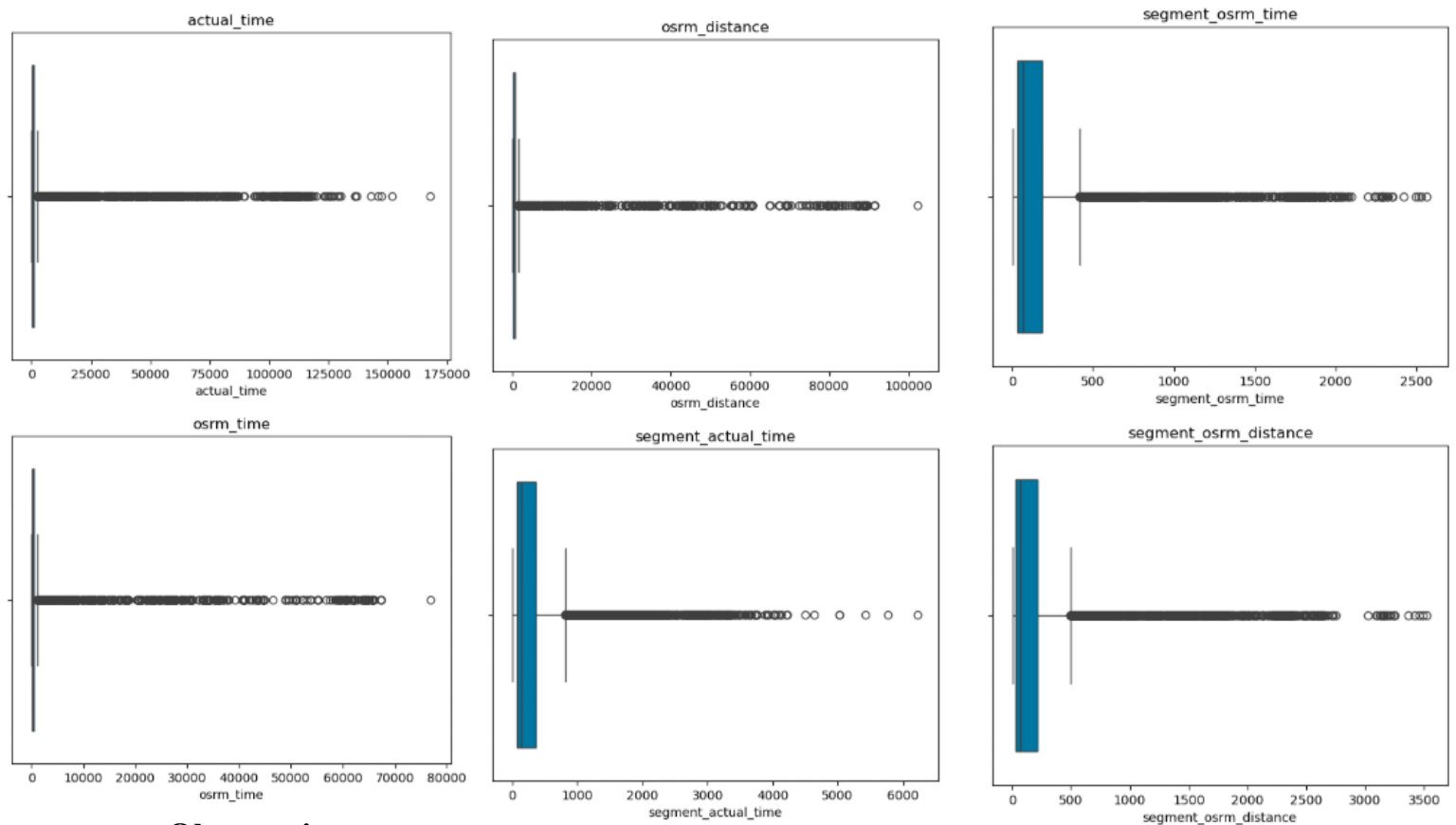
- Segment-level estimates closely matched aggregated route-level estimates.
- Data consistency was maintained throughout the aggregation process.

Business Insight

The aggregation strategy was successful and produced reliable trip-level metrics.

11. Distribution Analysis and Outlier Investigation

Boxplots were used to study numerical variables.



Observations

- Most time and distance variables exhibited strong right skewness.
- A small number of trips experienced extremely large delivery durations.
- Significant variability existed across shipments.

Insight

While most deliveries were completed within expected durations, a subset of shipments experienced substantial delays that warrant further operational investigation.

12. Outlier Treatment

```
#FIRSTLY COUNTING THE OUTLIERS

def count_outliers(df, col):
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1

    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    return df[(df[col] < lower) | (df[col] > upper)].shape[0]

for col in num_cols:
    print(col, count_outliers(trip_df, col))
```

```
actual_time 2069
osrm_time 1948
osrm_distance 2069
segment_actual_time 1643
segment_osrm_time 1492
segment_osrm_distance 1547
actual_distance_to_destination 1449
```

```
#NOW REMOVING THE OUTLIERS USING IQR METHOD
```

```
def remove_outliers(df, col):
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1

    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    return df[(df[col] >= lower) & (df[col] <= upper)]
```

```
#CALLING THE FUNCTION TO REMOVE OUTLIERS
```

```
trip_df_clean = trip_df.copy()

for col in num_cols:
    trip_df_clean = remove_outliers(trip_df_clean, col)
```

Outliers were identified using the Interquartile Range (IQR) method.

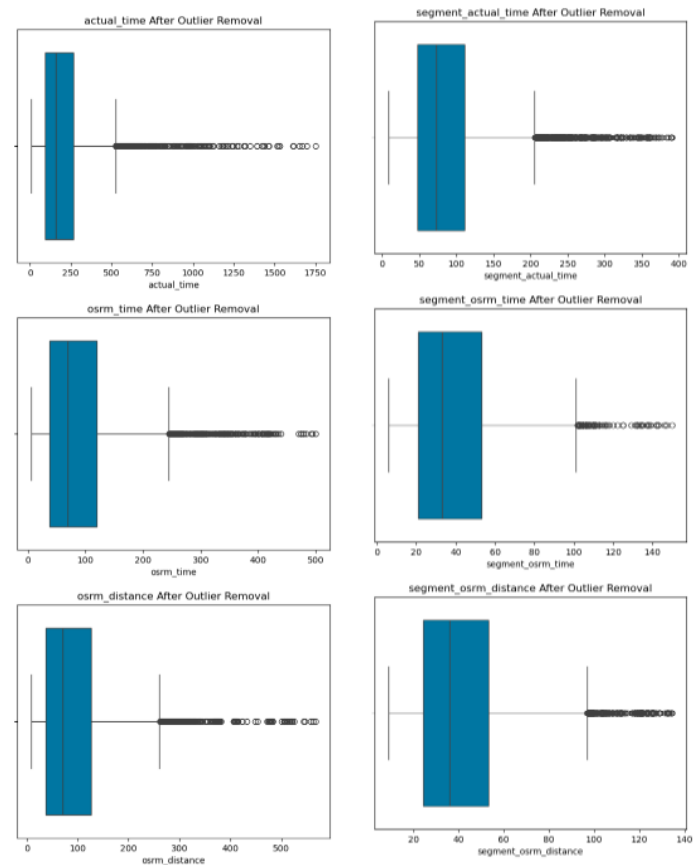
Given the highly skewed nature of the variables, IQR was selected instead of Z-score based approaches.

Observations

- Extreme observations were identified across almost all major numerical variables.
- Outlier treatment reduced the influence of unusual deliveries.

Insight

Removing extreme values improved dataset stability and prepared the data for future machine learning applications.



13. Categorical Encoding and Feature Scaling

Machine learning algorithms require numerical inputs.

```
HANDLING CATEGORICAL VARIABLES (ONE HOT ENCODING)

#IDENTIFYING CATEGORICAL VARIABLES
trip_df_clean.select_dtypes(include=['object','category']).columns

Index(['trip_uid', 'route_type', 'source_name', 'destination_name', 'weekday',
       'source_state', 'destination_state'],
      dtype='object')

#ENCODING route_type COLUMN
trip_df_clean['route_type'].value_counts() #checking no. of categories

route_type
Carting    7615
FTL        744
Name: count, dtype: int64

trip_df_encoded=pd.get_dummies(trip_df_clean,columns=['route_type'],drop_first=True)

trip_df_encoded.head()


```

	trip_uid	trip_creation_time	source_name	destination_name	actual_time	osrm_time	segment_actual_time	segment_osrm_time	osrm
1	trip-153671042288605164	2018-09-12 00:00:22.886430	Doddabtlpur_ChikaDPP_D (Karnataka)	Doddabtlpur_ChikaDPP_D (Karnataka)	399.0	210.0	141.0	65.0	
3	trip-153671046011330457	2018-09-12 00:01:00.113710	Mumbai Hub (Maharashtra)	Mumbai_MiraRd_JP (Maharashtra)	82.0	24.0	59.0	16.0	
5	trip-153671055416136166	2018-09-12 00:02:34.161600	Chennai_Poonamallee (Tamil Nadu)	Chennai_Poonamallee (Tamil Nadu)	92.0	30.0	60.0	23.0	
6	trip-153671066201138152	2018-09-12 00:04:22.011653	Chennai_Chrompet_DPC (Tamil Nadu)	Chennai_Vandalur_Dc (Tamil Nadu)	24.0	13.0	24.0	13.0	
7	trip-153671066076357165	2018-09-12 00:04:20.763577	HBR Layout PC (Karnataka)	HBR Layout PC (Karnataka)	122.0	65.0	64.0	34.0	

Encoding

One-Hot Encoding was applied to route_type.

Scaling

StandardScaler was applied to numerical variables.

STANDARDIZATION / NORMALIZATION

```
#USING STANDARD SCALER TO SCALE THE NUMERICAL FEATURES
num_cols

['actual_time',
 'osrm_time',
 'osrm_distance',
 'segment_actual_time',
 'segment_osrm_time',
 'segment_osrm_distance',
 'actual_distance_to_destination']

scaler=StandardScaler()
trip_df_encoded[num_cols]=scaler.fit_transform(trip_df_encoded[num_cols])

pd.set_option('display.float_format', lambda x: f'{x:.6f}')
trip_df_encoded[num_cols].describe()


```

	actual_time	osrm_time	osrm_distance	segment_actual_time	segment_osrm_time	segment_osrm_distance	actual_distance_to_destination
count	8359.000000	8359.000000	8359.000000	8359.000000	8359.000000	8359.000000	8359.000000
mean	0.000000	0.000000	-0.000000	-0.000000	0.000000	-0.000000	0.000000
std	1.000060	1.000060	1.000060	1.000060	1.000060	1.000060	1.000060
min	-1.143993	-1.184850	-1.154788	-1.354568	-1.495213	-1.387034	-1.280431
25%	-0.657145	-0.758014	-0.789824	-0.691012	-0.808908	-0.732768	-0.773434
50%	-0.270013	-0.317408	-0.347189	-0.265656	-0.259864	-0.227067	-0.248826
75%	0.357610	0.384807	0.374559	0.380886	0.655210	0.518679	0.526606
max	9.085679	5.616995	6.115474	5.144877	5.093316	4.020291	3.107617

Observations

- Categorical information became machine-learning compatible.
- Numerical features were standardized to comparable scales.

Insight

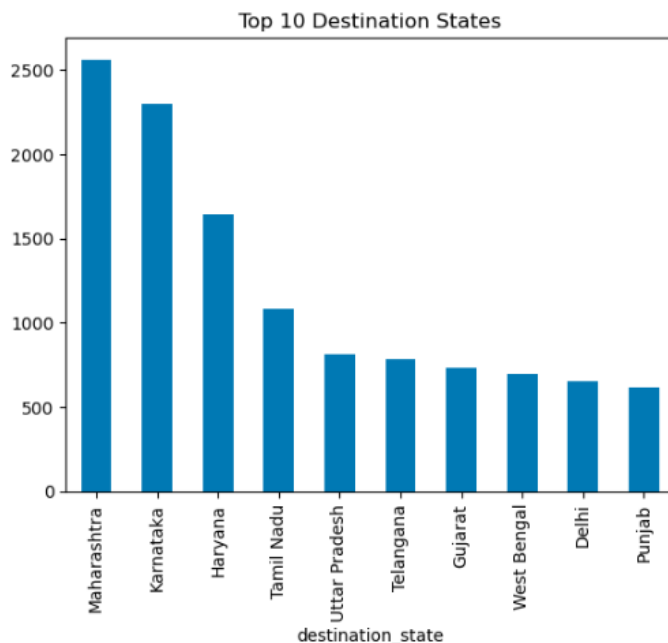
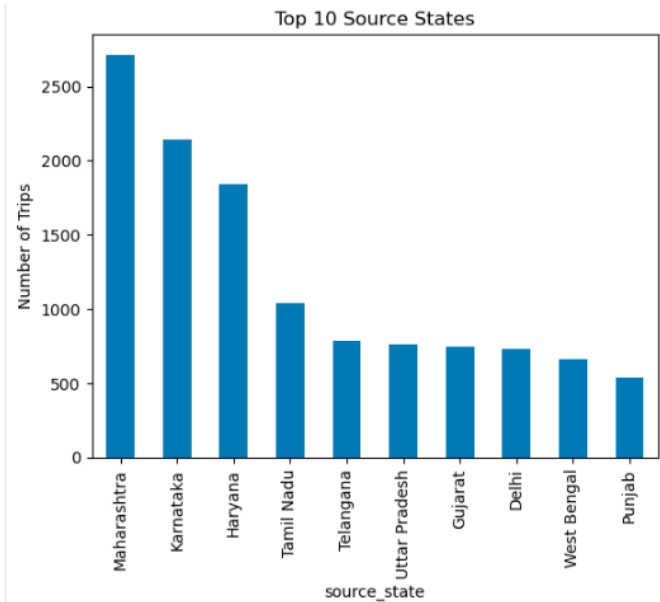
The final dataset became suitable for predictive modeling and advanced analytics.

14. Business Insights

Several important operational patterns emerged from the analysis.

Shipment Concentration

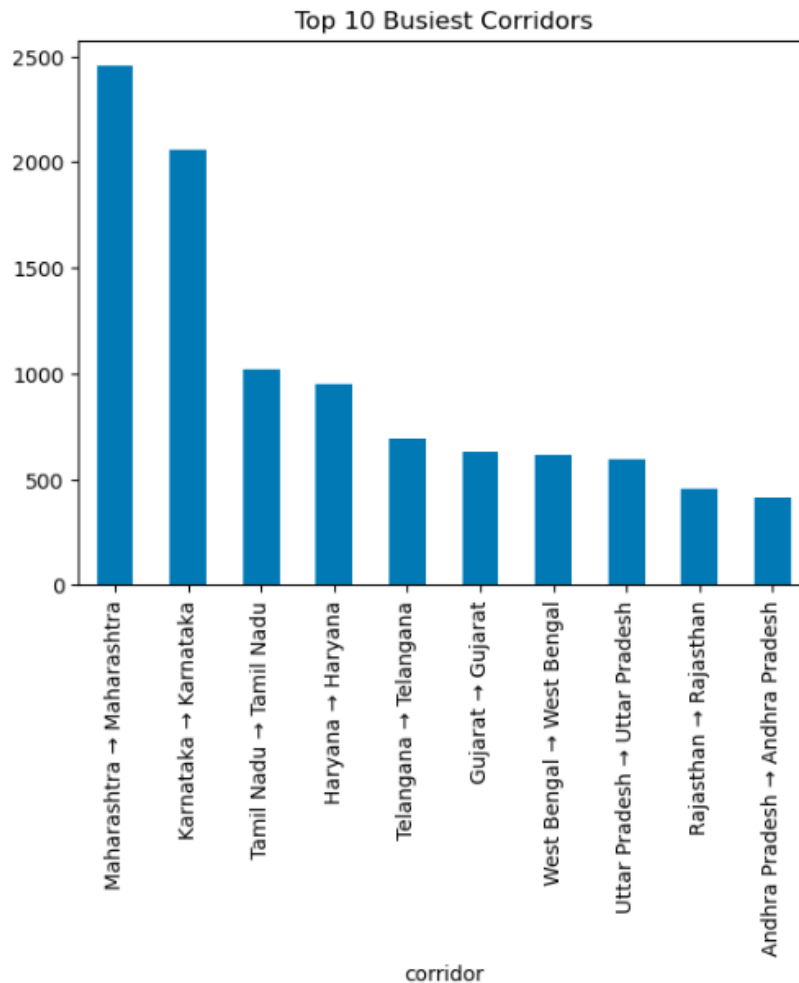
Maharashtra emerged as the largest logistics hub with **2,714 originating shipments**, followed by Karnataka (**2,143**) and Haryana (**1,838**).



On the destination side, Maharashtra again ranked first with **2,561 shipments**, followed by Karnataka (**2,294**) and Haryana (**1,643**).

This indicates that western and southern India represent the strongest shipment concentration zones within the network.

Corridor Analysis



The busiest corridors were primarily intra-state:

- Maharashtra → Maharashtra: **2,453 trips**
- Karnataka → Karnataka: **2,057 trips**

This suggests that a substantial portion of Delhivery's shipment volume is generated through regional distribution networks rather than long-haul interstate transportation.

Delivery Delays

Actual delivery duration averaged **4,078.8 minutes**, compared to an OSRM estimate of **2,092.2 minutes**. This represents a **95% increase** over estimated travel time and highlights the significant impact of operational processes on delivery performance.

High-Delay Corridors

```
#HIGHEST AVERAGE DELIVERY TIME WITHIN CORRIDORS (TOP 10)

corridor_time = (trip_df.groupby('corridor')['actual_time'].mean().sort_values(ascending=False))

corridor_time.head(10)

corridor
Punjab → Karnataka      136301.000000
Maharashtra → Assam      130199.000000
Delhi → Assam            126607.000000
Karnataka → Haryana      113721.285714
Assam → Delhi            111136.117647
Maharashtra → West Bengal 109489.333333
Haryana → Karnataka      105879.234043
Haryana → Tamil Nadu     105641.214286
Punjab → Maharashtra      82687.000000
Maharashtra → Jharkhand   82384.666667
Name: actual_time, dtype: float64
```

The longest average delivery durations were observed in:

- Punjab → Karnataka: **136,301 minutes**
- Maharashtra → Assam: **130,199 minutes**
- Delhi → Assam: **126,607 minutes**

These corridors represent the most critical operational bottlenecks identified during the analysis.

15. Recommendations

Based on the analysis, the following recommendations are proposed:

- Enhance ETA prediction systems by incorporating historical corridor-level delays, hub processing times, and operational waiting periods.
 - Prioritize operational audits for Punjab → Karnataka, Maharashtra → Assam, and Delhi → Assam, as these corridors exhibit the highest average delivery durations.
 - Establish corridor-specific SLA targets rather than applying uniform service benchmarks across all routes.
 - Develop a shipment-delay monitoring dashboard focused on high-risk routes and recurring outlier deliveries.
 - Use the engineered dataset to build predictive delay models capable of identifying at-risk shipments before SLA breaches occur.
 - Introduce corridor-level performance benchmarking to continuously measure route efficiency and identify emerging operational bottlenecks.
-

16. Conclusion

This project successfully transformed **144,867 segment-level shipment records into 14,807 trip-level observations**, creating a structured dataset suitable for advanced analytics and machine learning.

Statistical analysis revealed that actual delivery times are approximately **95% higher than OSRM route estimates**, demonstrating that operational factors play a critical role in shipment performance.

Correlation analysis confirmed the validity of the aggregation methodology, with route-level and segment-level metrics exhibiting correlations above **0.90**.

The study identified Maharashtra, Karnataka, and Haryana as the most significant shipment hubs and highlighted critical bottlenecks in long-distance interstate corridors such as Punjab → Karnataka and Maharashtra → Assam.

Overall, the project demonstrates how feature engineering, statistical validation, and logistics analytics can be leveraged to improve ETA prediction, route optimization, and operational efficiency across large-scale transportation networks.

The resulting dataset provides a strong foundation for future forecasting, route optimization, and machine learning initiatives within the logistics domain.